

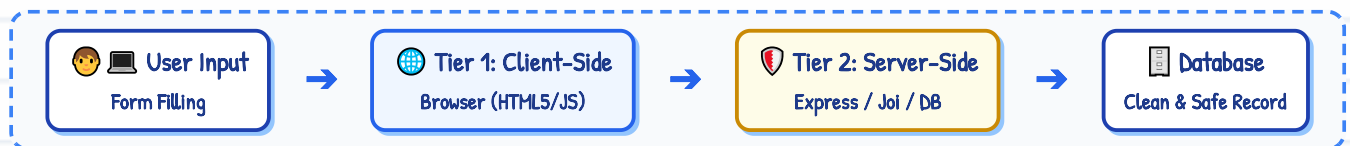
TOPIC 01: FORM VALIDATION & 2-TIER DEFENSE SYSTEM

Form Validation Kya Hai? (Core Definition)

Form Validation ka matlab hai user dwara form me enter kiye gaye data ko **check karna** ki wo correct format me hai, required rules (length, email pattern, range) ko follow karta hai, aur kisi malicious/harmful code se safe hai ya nahi.

The 2-Tier Defense System (Validation Architecture):

Web applications me validation hamesha do alag-alag layers (tiers) par kaam karti hai:



Dono Validation Layers Ka Role:

- **Tier 1 (Client-Side Validation):** Browser ke andar execute hoti hai. User ko turant (instant) pata chalta hai ki usne email galat likha hai ya password chhota hai. Yeh **User Experience (UX)** aur network bandwidth bachane ke liye best hai.
- **Tier 2 (Server-Side Validation):** Backend server (Node.js/Express) par chalti hai. Database me save hone se pehle data ko verify karti hai. Yeh **Security aur Data Integrity** ke liye zaroori hai.

Golden Security Rule: "Never Trust Client Input!"



Client-side validation ko **Postman**, **cURL**, ya **Browser Inspect (F12)** se JavaScript disable karke 1 second me bypass kiya ja sakta hai! Isliye **Server-Side validation HAMESHA MANDATORY** hoti hai!

TOPIC 02: CLIENT-SIDE vs SERVER-SIDE VALIDATION

Master Comparison Table (Exam & Interview Special!):

Feature / Parameter	Client-Side Validation	Server-Side Validation
Where It Runs	User ke browser me (Frontend: HTML5, JS, React).	Web Server par (Backend: Node.js, Express, Python).
Primary Purpose	User Experience (UX) & instant visual feedback.	Security, Data Integrity & Business Logic.
Response Time / Speed	Instant (0 ms); bina network call ke turant error dikhata hai.	Thoda slow; Network round-trip (HTTP request/response) lagta hai.
Can It Be Bypassed?	YES! Very easily (Postman, cURL, Inspect element).	NO! Cannot be bypassed (Server enforces it).
Network Traffic	Network traffic bachata hai (invalid data bhejta hi nahi).	Har request par network data consume hota hai.
Database Access	No DB Access: Yeh check nahi kar sakta ki email already exist karta hai ya nahi.	Full DB Access: Database query karke duplicate records check kar sakta hai.
Tools & Technologies	HTML5 (required, pattern), JS Constraint API, Bootstrap.	Joi, express-validator, Yup, Zod, Mongoose Schema.

Topper Insight:



Dono ek dusre ke competitor nahi, balki **partners** hain! Client-side user ko khush rakhta hai, aur Server-side application aur database ko safe rakhta hai!

TOPIC 03: ALL TYPES OF FORM ERRORS (INPUT ERRORS)

Category 1: Input & Data Constraint Errors

Form me aane wale sabhi standard input errors jo client aur server dono par check hote hain:

- **1. Missing / Required Field Error:**

User ne mandatory field ko khali (empty) chhod diya (e.g. `value.trim() === ""`).

Error: "Username is required!" ya "Please fill out this field."

- **2. Type & Format Mismatch Error:**

Field me galat data type ya wrong format enter kiya gaya hai:

- *Email Format:* @ ya domain missing hona (e.g. `shivamgmail.com`).
- *Phone Number:* 10 digits ki jagah letters daal dena.
- *URL Format:* `https://` missing hona.

- **3. Length Constraint Error (Min / Max Length):**

Text ki character length limit se bahar hona:

- *Password too short:* Minimum 8 characters chahiye par user ne 4 likhe.
- *Bio / Message too long:* Maximum 500 characters ki limit cross ho gayi.

- **4. Range & Boundary Error (Min / Max Value):**

Numeric fields ya Dates ka valid boundary se bahar hona:

- *Age limit:* Age < 18 ya > 120 enter karna.
- *Negative price:* Product price - 250 daal dena.
- *Future Date of Birth:* Janam ki taareekh aane wale saal ki daal dena.

- **5. Pattern / Regex Mismatch Error:**

Complex pattern match na hona (e.g. Strong Password policy: At least 1 uppercase, 1 number, 1 special character).

- **6. Value Confirmation / Matching Error:**

Do fields ka aapas me match na hona (e.g. `Password !== Confirm Password`).

- **7. File Upload Errors:**

- *File size exceeded:* Uploaded image > 5MB limit.
- *Invalid MIME type:* .exe ya .bat upload karna jab sirf .jpg / .png allowed hai.

TOPIC 04: BUSINESS, AUTH & SYSTEM ERRORS

Category 2: Business Logic & Semantic Errors (Server-Only)

- **8. Duplicate / Uniqueness Error (Unique Key Constraint):**
Data format sahi hai, par database me pehle se registered hai!
Error: "Email already registered! Please login." ya "Username 'shivam_dev' already taken."
- **9. State / Availability Conflict Error:**
Resource ab available nahi hai (e.g. "Seat already booked by another user", "Product out of stock").

Category 3: Authentication, Authorization & Security Errors

- **10. Invalid Credentials Error:**
Login form me password ya email galat enter karna (HTTP 401 Unauthorized).
- **11. CSRF Token Mismatch Error:**
Cross-Site Request Forgery security token invalid ya missing hona (HTTP 403 Forbidden).
- **12. CAPTCHA Verification Failure:**
Google reCAPTCHA solve na hona (Bot protection trigger hona).
- **13. Session Expired Error:**
Form bharte-bharte user ka login token expire ho jana.

Category 4: Technical, Network & HTTP Protocol Errors

- **14. HTTP 400 Bad Request:** Form body corrupt hona ya JSON parser error.
- **15. HTTP 429 Too Many Requests (Rate Limiting):** Spam prevention—user ne 1 minute me 20 baar login submit kiya!
- **16. HTTP 500 Internal Server Error:** Backend me Database down ho jana ya server code crash ho jana.
- **17. Network Offline / Timeout Error:** User ka internet disconnect ho jana form submit karte waqt.

TOPIC 05: CLIENT-SIDE VALIDATION (HTML5 & BOOTSTRAP)

HTML5 Native Validation Attributes:

Browser me bina JS ke bhi form check ho sakta hai in attributes se:

- `required`: Field khali nahi chhod sakte.
- `type="email"` / `type="number"`: Automatic type check.
- `minlength="6"` / `maxlength="20"`: Character limit.
- `min="1"` / `max="100"`: Number ki range limit.
- `pattern="[A-Za-z0-9]{5,}"`: Custom Regular Expression (Regex).

Bootstrap 5 Custom Validation Mechanism:

Browser ke ugly default popups ko disable karne ke liye Bootstrap 5 ka standard pattern use karte hain:

- Form me `class="needs-validation"` `novalidate` add karte hain.
- Har input ke niche `<div class="valid-feedback">Looks good!</div>` aur `<div class="invalid-feedback">Please enter valid title!</div>` lagate hain.

```
public/js/client-validation.js (Bootstrap 5 Standard)

// Fetch all forms with 'needs-validation' class
const forms = document.querySelectorAll('.needs-validation');

Array.from(forms).forEach(form => {
  form.addEventListener('submit', event => {
    // checkValidity() evaluates all HTML5 constraints
    if (!form.checkValidity()) {
      event.preventDefault();
      event.stopPropagation();
    }
    form.classList.add('was-validated');
  }, false);
});
```



Mechanism: `was-validated` class lagte hi valid fields green ho jaati hain aur invalid fields red border aur error message ke saath highlight ho jaati hain!

TOPIC 06: SERVER-SIDE VALIDATION WITH JOI

Server Par Joi Package Kyu Use Karte Hain?

Agar koi hacker ya tester Postman se request bhejta hai ya browser JS disable kar deta hai, toh HTML5 validation completely bypass ho jaati hai! Server par har request body ko strictly validate karne ke liye **Joi** library use hoti hai.

Installation: npm install joi

Joi Schema Definition & Express Middleware:

```
schema.js & middleware/validate.js

const Joi = require('joi');
const ExpressError = require('../utils/ExpressError');

// 1. Define strict schema rules
const postSchema = Joi.object({
  post: Joi.object({
    title: Joi.string().required().min(3).max(100),
    content: Joi.string().required(),
    author: Joi.string().required()
  }).required()
}).required();

// 2. Express Validation Middleware
const validatePost = (req, res, next) => {
  const { error } = postSchema.validate(req.body);
  if (error) {
    const errMsg = error.details.map(el => el.message).join(', ');
    throw new ExpressError(400, errMsg);
  } else {
    next(); // Passed validation! Proceed to route handler
  }
};
```



Benefit of Middleware: Route handler me bar-bar if (!title) likhne ki zaroorat nahi padti. Route se pehle hi invalid requests block ho jaate hain!

TOPIC 07: CUSTOM ERROR CLASS & WRAPASYNC

1. Custom ExpressError Class:

JavaScript ki standard built-in Error class me HTTP statusCode store karne ka tareeqa nahi hota. Isliye hum custom class banate hain:

```
utils/ExpressError.js

class ExpressError extends Error {
  constructor(statusCode, message) {
    super();
    this.statusCode = statusCode;
    this.message = message;
  }
}

module.exports = ExpressError;
```

2. wrapAsync Utility (Catching Async Errors):

Express me jab asynchronous function (Database call) fail hota hai, toh app crash na ho isliye har jagah try-catch lagana padta hai. Is code repetition ko khatam karta hai **wrapAsync**:

```
utils/wrapAsync.js

// Higher-Order Function that passes rejected promises to next()
const wrapAsync = (fn) => {
  return (req, res, next) => {
    fn(req, res, next).catch(next);
  };
};

module.exports = wrapAsync;
```



How it works: `app.post('/posts', wrapAsync(async (req, res) => { ... })).` Agar koi DB error aayega toh wrapAsync automatically error ko global handler middleware tak forward kar dega!

TOPIC 08: GLOBAL ERROR MIDDLEWARE & USER ERROR UI

Express Global Error-Handling Middleware:

Express server me saare routes ke baad sabse aakhri me lagaya jata hai (4 parameters (err, req, res, next) ke saath):

```
server.js (Global Error Catch-All & Handler)

// 1. Catch-All for Undefined Routes (404 Not Found)
app.all('*', (req, res, next) => {
  next(new ExpressError(404, "Page Not Found!"));
});

// 2. Global Error Handling Middleware (4 arguments required)
app.use((err, req, res, next) => {
  const { statusCode = 500, message = "Something went wrong!" } = err;
  res.status(statusCode).render("error.ejs", { statusCode, message, err });
});
```

User-Friendly Error Page Template (views/error.ejs):

```
views/error.ejs

<div class="row mt-5">
  <div class="col-6 offset-3">
    <div class="alert alert-danger" role="alert">
      <h4 class="alert-heading">Error <%= statusCode %>!/h4>
      <p><%= message %></p>
    </div>
  </div>
</div>
```



Production Golden Rule: Production environment me users ko kabhi bhi ugly raw database stack traces mat dikhao! Clean polite error UI dikhao, aur server logs me full error trace monitor karo.